1. Create the shop database and the products table exactly as shown.
Confirm with SHOW TABLES;.

```
CREATE DATABASE students;
USE students;

CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    age INT,
    address VARCHAR(100)
);

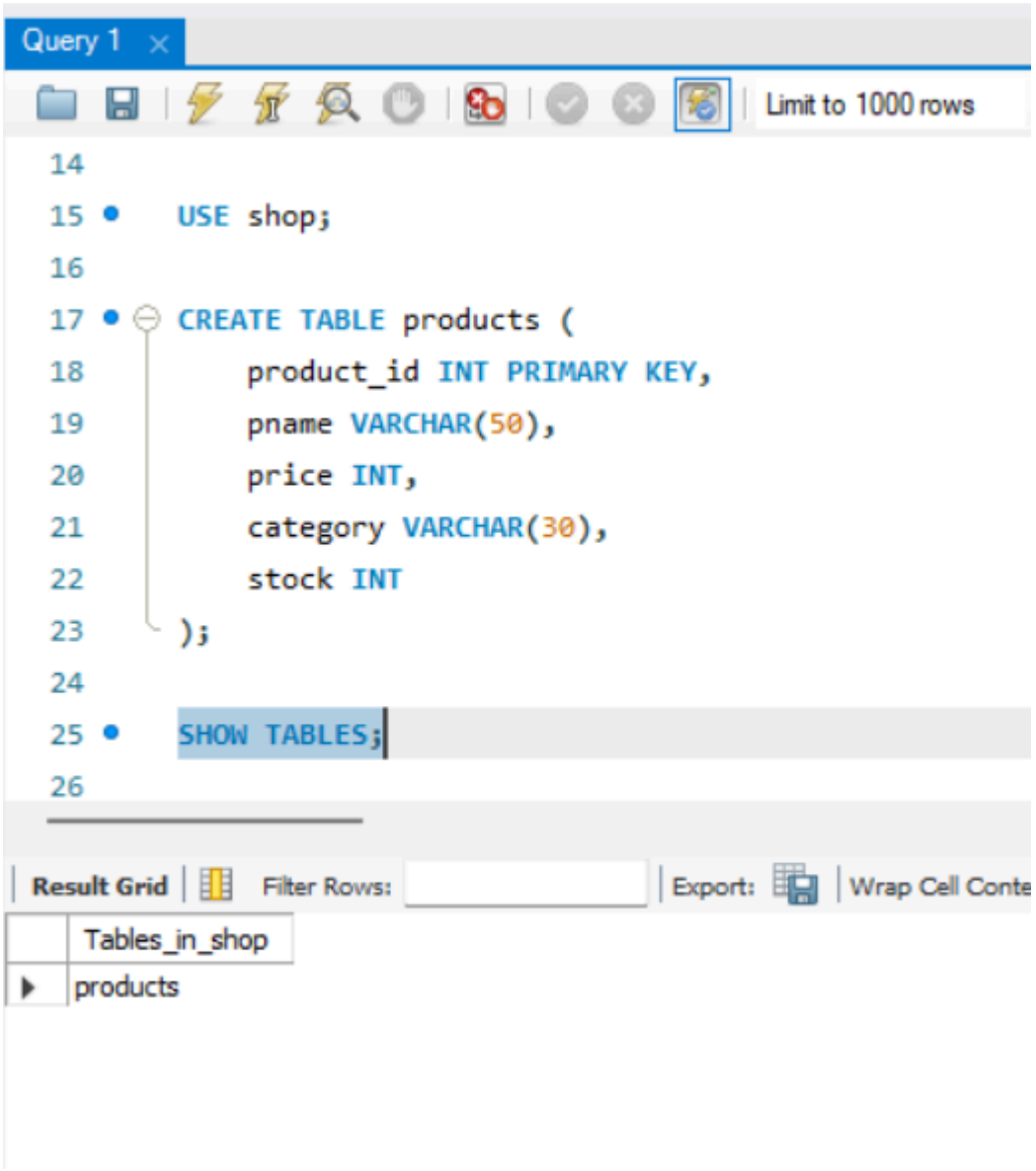
SELECT * FROM students;

CREATE DATABASE shop;

USE shop;

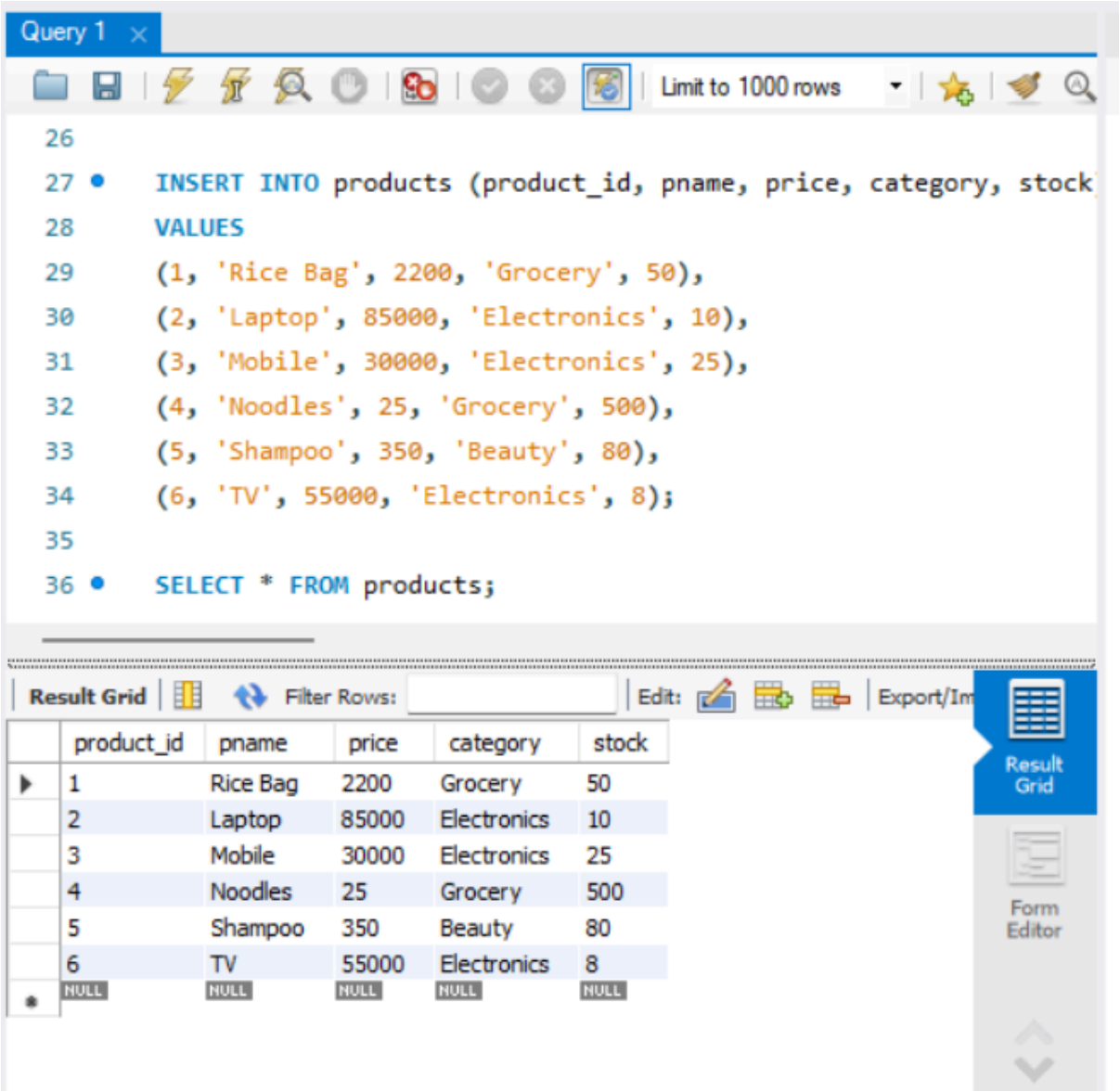
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    pname VARCHAR(50),
    price INT,
    category VARCHAR(30),
    stock INT
);

SHOW TABLES;
```



2. Insert all 6 products in one single INSERT statement.

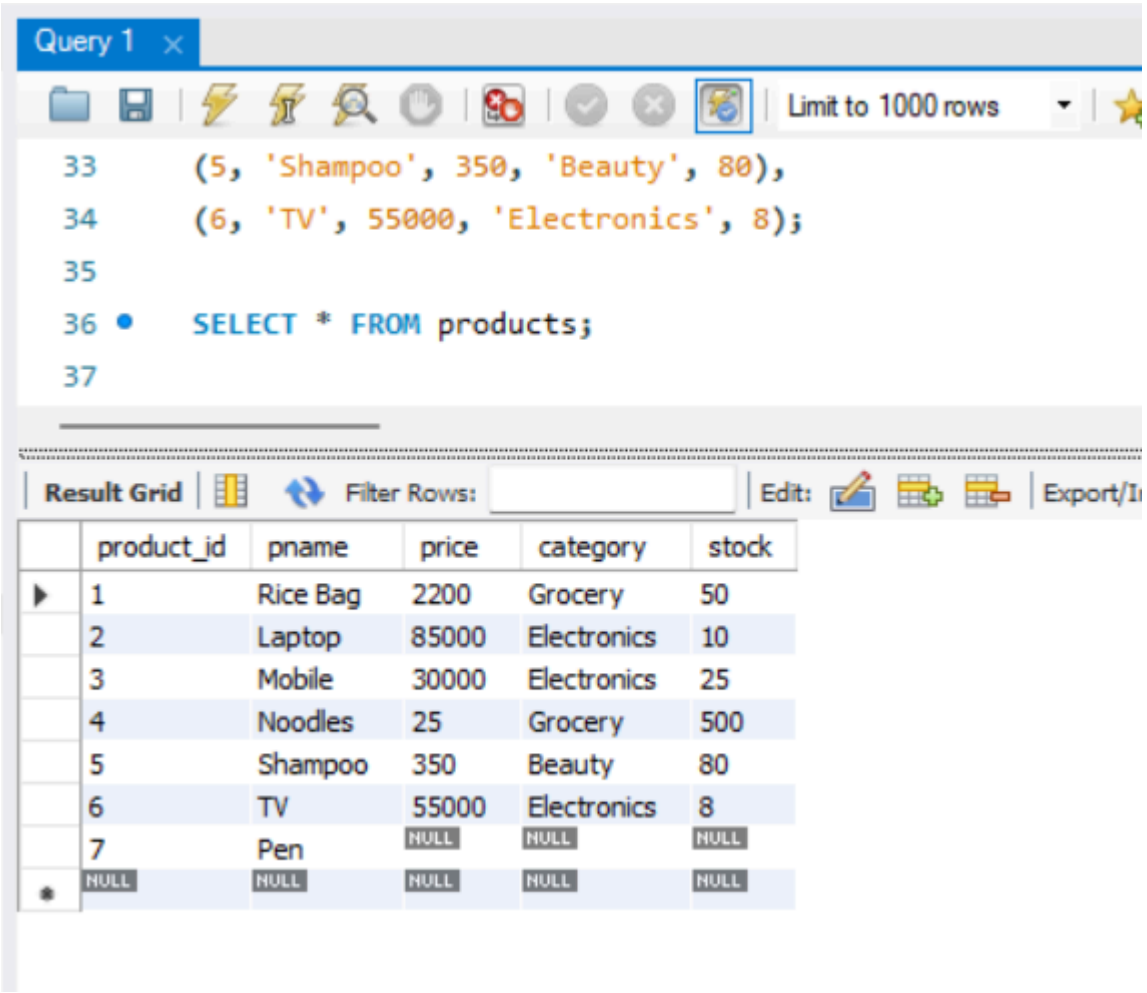
INSERT INTO products (product_id, pname, price, category, stock)
VALUES
(1, 'Rice Bag', 2200, 'Grocery', 50),
(2, 'Laptop', 85000, 'Electronics', 10),
(3, 'Mobile', 30000, 'Electronics', 25),
(4, 'Noodles', 25, 'Grocery', 500),
(5, 'Shampoo', 350, 'Beauty', 80),
(6, 'TV', 55000, 'Electronics', 8);



3. Insert product 7, giving only product id and pname (’Pen’). Which columns become NULL?

SELECT * FROM products;

INSERT INTO products (product_id, pname)
VALUES (7, 'Pen');



4. Try to insert another product with product id = 3. Write down the exact error and explain why it happened.

INSERT INTO products
VALUES (3, 'Mobile', 30000, 'Electronics', 25);

The error occurs because `product_id = 3` already exists in the `products` table, and `product_id` is the Primary Key. A primary key must contain unique values, so MySQL does not allow another product with the same ID.

5. Insert two more products of your choice in one statement: one Grocery item and one Beauty item.

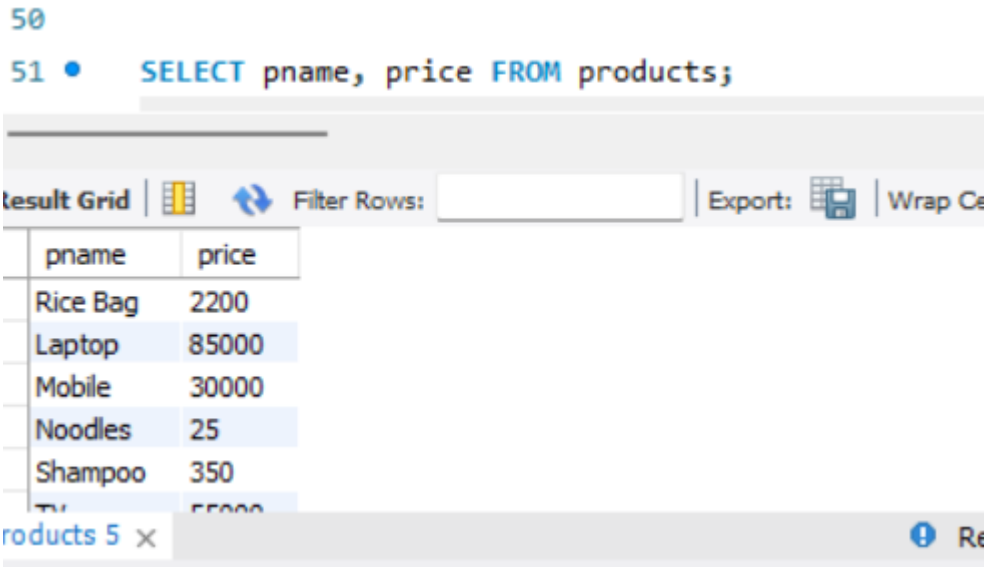
INSERT INTO products (product_id, pname, price, category, stock)
VALUES
(8, 'Biscuits', 100, 'Grocery', 60),
(9, 'Face Wash', 450, 'Beauty', 40);

6. Show all columns of all products.

SELECT * FROM products;

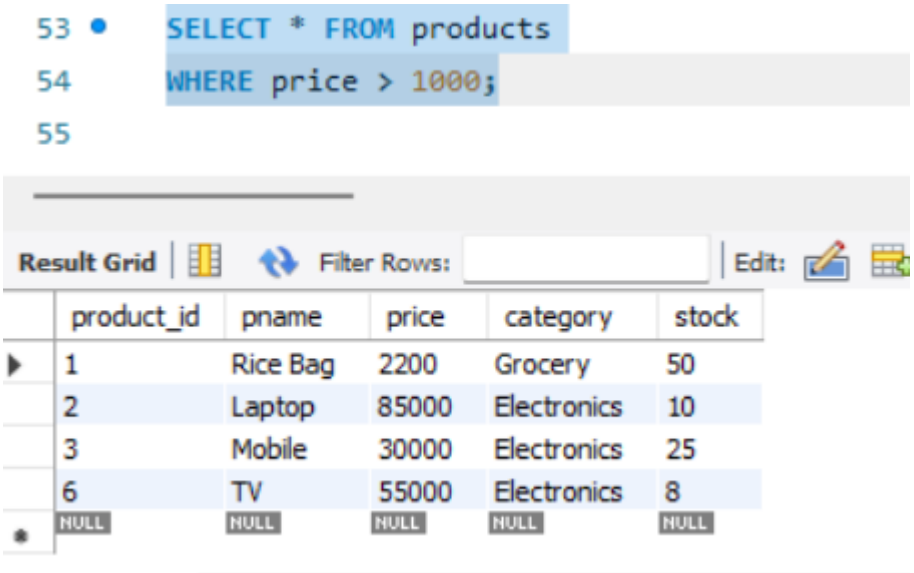
7. Show only pname and price.

SELECT pname, price FROM products;



8. Show products with price above 1000.

SELECT * FROM products
WHERE price > 1000;



9. Show Electronics products with stock less than 20 (AND).

SELECT * FROM products
WHERE category = 'Electronics' AND stock < 20;

```
56 •  SELECT * FROM products
57    WHERE category = 'Electronics' AND stock < 20;
58
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
2	Laptop	85000	Electronics	10
6	TV	55000	Electronics	8
NULL	NULL	NULL	NULL	NULL

10. Show products that are Grocery or cost less than 500 (OR).

SELECT * FROM products
WHERE category = 'Grocery' OR price < 500;

```
59 •  SELECT * FROM products
60    WHERE category = 'Grocery' OR price < 500;
61
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
1	Rice Bag	2200	Grocery	50
4	Noodles	25	Grocery	500
5	Shampoo	350	Beauty	80
NULL	NULL	NULL	NULL	NULL

11. Show products in categories Grocery and Beauty, using IN.

SELECT * FROM products
WHERE category IN ('Grocery', 'Beauty');

```
62 •  SELECT * FROM products
63    WHERE category IN ('Grocery', 'Beauty');
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
1	Rice Bag	2200	Grocery	50
4	Noodles	25	Grocery	500
5	Shampoo	350	Beauty	80
NULL	NULL	NULL	NULL	NULL

12. Show products with price between 300 and 40000. Is 350 included? Why?

SELECT * FROM products
WHERE price BETWEEN 300 AND 40000;

- `WHERE` filters products based on a condition.
- `BETWEEN 300 AND 40000` selects prices from 300 to 40000, including both endpoints.
- Yes, 350 is included because it falls between 300 and 40000.

Conclusion: The `BETWEEN` operator in SQL is inclusive, meaning it includes both the minimum and maximum values.

```
65 •  SELECT * FROM products
66    WHERE price BETWEEN 300 AND 40000;
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
1	Rice Bag	2200	Grocery	50
3	Mobile	30000	Electronics	25
5	Shampoo	350	Beauty	80
NULL	NULL	NULL	NULL	NULL

13. Show products whose name starts with M.

SELECT * FROM products

WHERE pname LIKE 'M%';

```
68 •  SELECT * FROM products
69    WHERE pname LIKE 'M%';
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
3	Mobile	30000	Electronics	25
NULL	NULL	NULL	NULL	NULL

14. Show products whose name contains the letter o.

SELECT * FROM products
WHERE pname LIKE '%o%';

```
71 •  SELECT * FROM products
72    WHERE pname LIKE '%o%';
```

Result Grid | Filter Rows: | Edit:

product_id	pname	price	category	stock
2	Laptop	85000	Electronics	10
3	Mobile	30000	Electronics	25
4	Noodles	25	Grocery	500
5	Shampoo	350	Beauty	80
NULL	NULL	NULL	NULL	NULL

15. Show the 2 most expensive products (which two keywords together?).

SELECT * FROM products
ORDER BY price DESC
LIMIT 2;

```
74 •  SELECT * FROM products
75    ORDER BY price DESC
76    LIMIT 2;
```

Result Grid | Filter Rows: | Edit: | Exp

product_id	pname	price	category	stock
2	Laptop	85000	Electronics	10
6	TV	55000	Electronics	8
NULL	NULL	NULL	NULL	NULL

The two keywords used together are `ORDER BY` and `LIMIT`. `DESC` is also used to show the most expensive products first.

16. The Laptop price dropped to 79000. Update it using its id.

SELECT * FROM products
WHERE pname = 'Laptop';

```
78 •  SELECT * FROM products
79    WHERE pname = 'Laptop';
```

Result Grid | Filter Rows: | Edit:

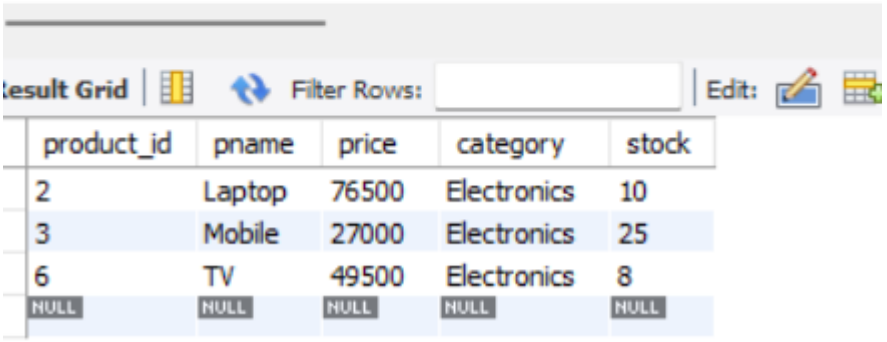
product_id	pname	price	category	stock
2	Laptop	85000	Electronics	10
NULL	NULL	NULL	NULL	NULL

The `UPDATE` statement changes the Laptop's price to 79000. The `WHERE` clause ensures that only the product with ID 2 is updated.

17. Festival offer: reduce the price of every Electronics product by 10%.
Hint: SET price = price * 0.9.

SET SQL_SAFE_UPDATES = 0;
UPDATE products
SET price = price * 0.9
WHERE category = 'Electronics';

```
86 •    SET SQL_SAFE_UPDATES = 0;
87 •    UPDATE products
88      SET price = price * 0.9
89      WHERE category = 'Electronics';
90
91 •    SELECT * FROM products
92      WHERE category = 'Electronics';
93
```



product_id	pname	price	category	stock
2	Laptop	76500	Electronics	10
3	Mobile	27000	Electronics	25
6	TV	49500	Electronics	8
NULL	NULL	NULL	NULL	NULL

18. Ten Rice Bags were sold. Decrease its stock by 10 with one query.

```
 UPDATE products
SET stock = stock - 10
WHERE pname = 'Rice Bag';
```

19. Delete the product named Pen. Then delete all products with stock below 10. How many rows did each query remove?

```
DELETE FROM products
WHERE pname = 'Pen';

DELETE FROM products
WHERE stock < 10;
```

`DELETE FROM` removes rows from the table.
`WHERE pname = 'Pen'` deletes the product named Pen.
`WHERE stock < 10` deletes all products whose stock is less than 10.

20. Run SELECT to check the final table. Then answer: which command would empty the table but keep it, and which would remove the table completely?

```
SELECT * FROM products;
TRUNCATE TABLE products;
```

```
DROP TABLE products;
```

This displays all remaining products after the deletion queries.